

micro:bit

Introduction to Computer Science

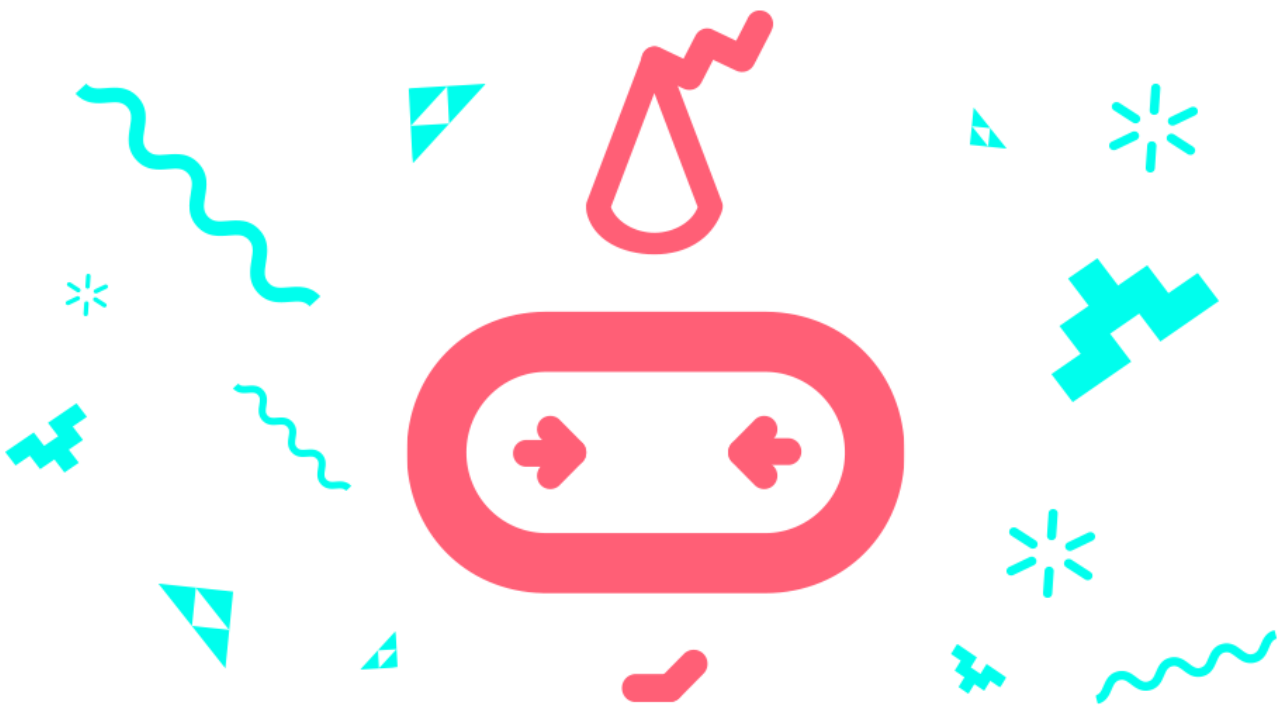
Unit 5: Iteration

Student workbook
makecode.microbit.org



Table of Contents

Overview	3
Unit summary	3
Learning goals	3
Lesson A: Understanding iteration	4
Lesson B: Coding with loops	5
Lesson C: Get loopy!	20
Glossary of key terms	25



Overview

Unit summary

This unit introduces the concept of iteration—or ways to make things repeat. In MakeCode, this is accomplished with loop blocks. In an unplugged activity, you will look for opportunities to use repetition to complete a task, and you will start to see the benefits of using repeat loops in programming. In the coding activities, you'll learn to code with three types of loop blocks as well as sprite and music blocks. In the project, you will code your own unique program using loops, variables, and other blocks you've explored and learned. You'll design and build an object that uses sound, display, and motion in some way. To incorporate sound and motion, you may be able to use additional available materials such as micro-servo motors (a small motor), crocodile clips, headphones, and speakers for Lessons B and C of this Unit.

Learning goals

During this unit, you will:

- Understand the value of iteration in programming.
- Understand looping as a form of iteration.
- Learn how and when to use the looping blocks 'repeat', 'while', and 'for'.
- Apply the above knowledge and skills to create a unique program that uses iteration and looping as an integral part of the project.

Lesson A: Understanding iteration

Overview: Iteration

In computer programming, *iteration* is the repetition of a sequence of commands. Computer programmers can use a special type of code called a loop around the commands they want to repeat as a form of iteration. A *loop* repeats code until a certain condition is met.

Lesson B: Coding with loops

Overview: Introduce three types of loops in Microsoft MakeCode

For today's coding activities, you'll use three different loop blocks in Microsoft MakeCode:

- 'repeat' block – This block repeats the code n number of times.
- 'while' block – This block runs the code as long as the condition inside of it is true.
- 'for' block – This block repeats the code n number of times, but with a variable.

We'll do three coding activities to demonstrate how each type of loop block works.

Coding activity 1: Code a sprite to walk a square with the 'repeat' loop

To start, you can code the same algorithm you created in the [Lesson A, unplugged activity](#):

Step forward

Turn left

Step forward

Turn left

Step forward

Turn left

Step forward

Turn left

Only, in this case, you will be coding a "sprite" or LED light to move around on your micro:bit.

1. In Microsoft MakeCode, start a new project and name it something like: **Sprite walking square**. You can leave the 'on start' block in the coding Workspace but can delete the 'forever' loop block.

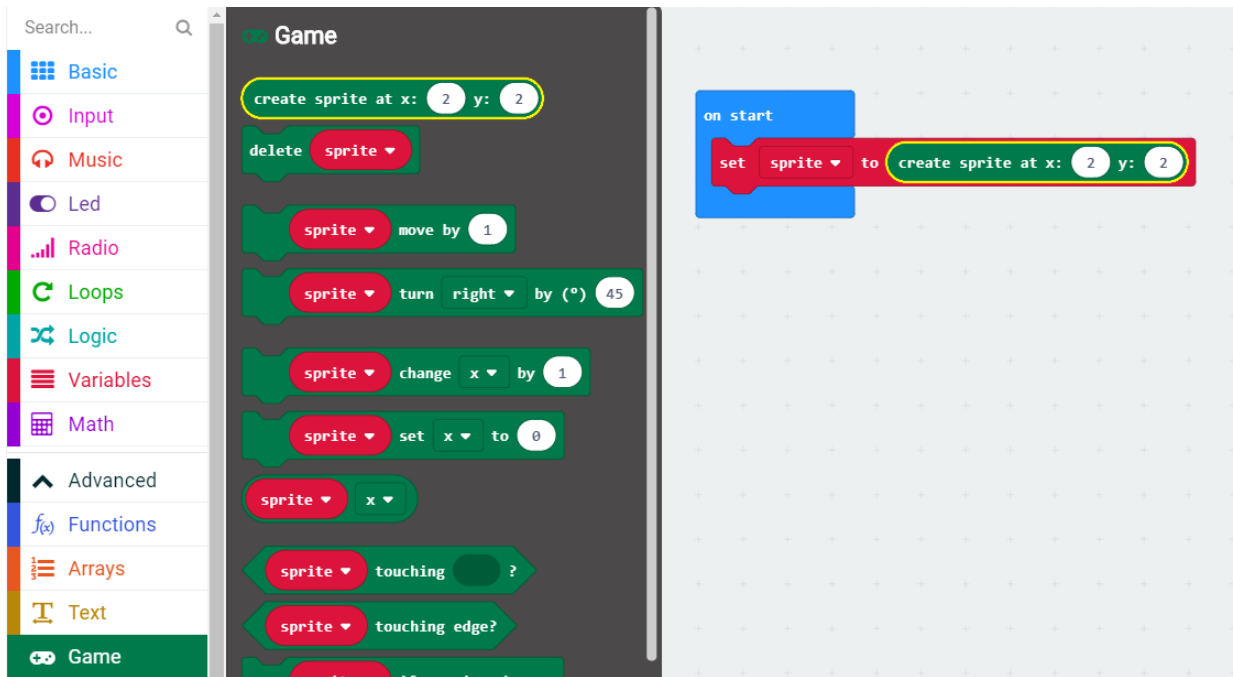
Create a Sprite Variable

2. On start, you want the sprite to appear. To make this happen, go to the Variables Toolbox drawer, and select the "Make a Variable" button. Name the new variable **sprite** and select OK. Now, in the Variables Toolbox drawer, drag a 'set sprite to' block to the coding Workspace and drop it inside the 'on start' block.

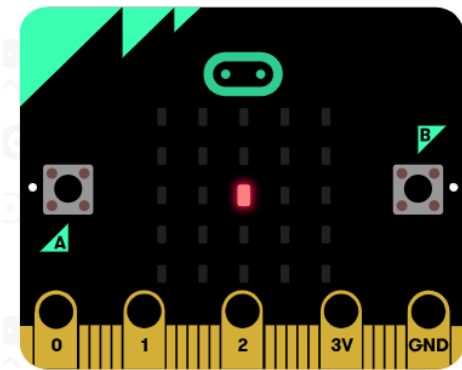


Create a sprite

3. Select the Advanced category at the bottom of the Toolbox. This will open up more of the Toolbox menu. Select the Game category and drag a 'create sprite' oval block to the coding Workspace. Drop it in the 'set sprite' block, replacing 0.



You should now see the 'sprite', or a red LED light appear in the middle of the micro:bit simulator.



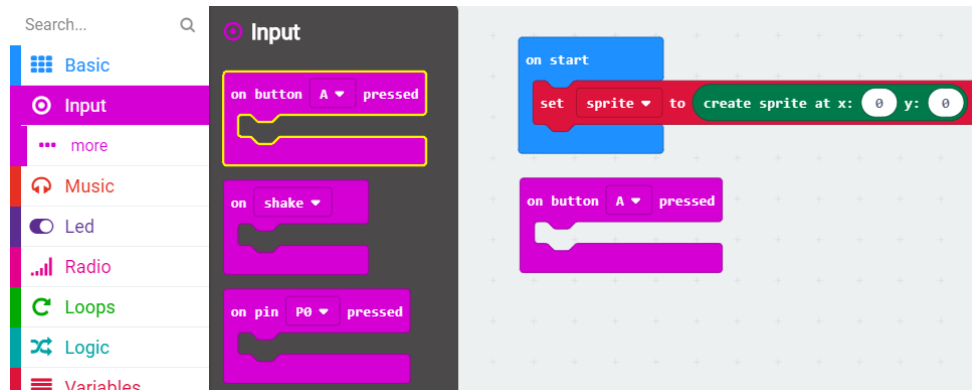
On the face of the micro:bit is a 5 x 5 grid of LED lights. The X coordinates are the horizontal light positions that go from 0-4, and the Y coordinates are the vertical light positions that go from 0-4 as well. You can see from the code blocks that we've created our sprite at X, Y position (2, 2). If we want to start our sprite in the top left of the screen, we'll have to change the starting coordinates to (0, 0).



Moving the sprite

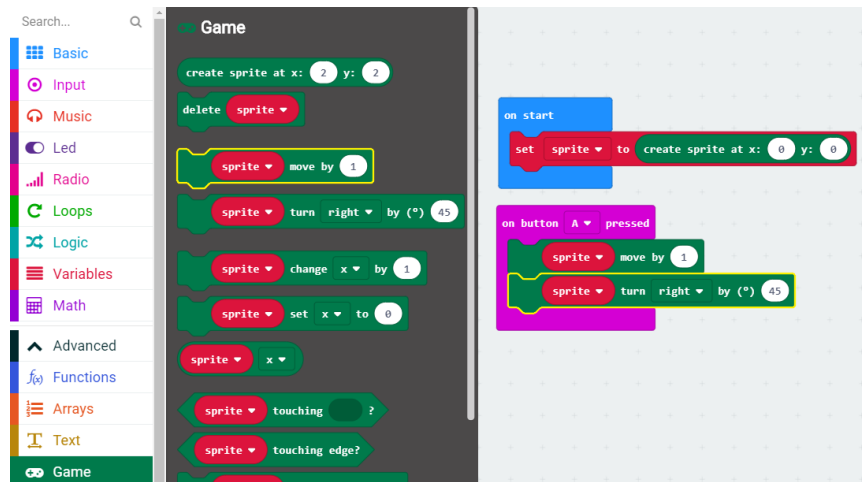
Now let's make our sprite move around the face of the micro:bit. You'll activate this when you press a button.

4. From the Input Toolbox drawer, drag an 'on button pressed' block onto the workspace.



You'll need two more blocks from the Game menu. Referring to the pseudocode, see if you can find the blocks you need for moving their sprite forward and turning their sprite.

5. From the Game Toolbox drawer, drag out a 'sprite move by' block and a 'sprite turn right by' block to the coding Workspace. Drop into the 'on button pressed' block.



6. To make the sprite move from one side of the screen to the other (as though walking around a chair), you'll need to move the sprite 4 places. So, change the value in the 'sprite move by' block from 1 to 4.



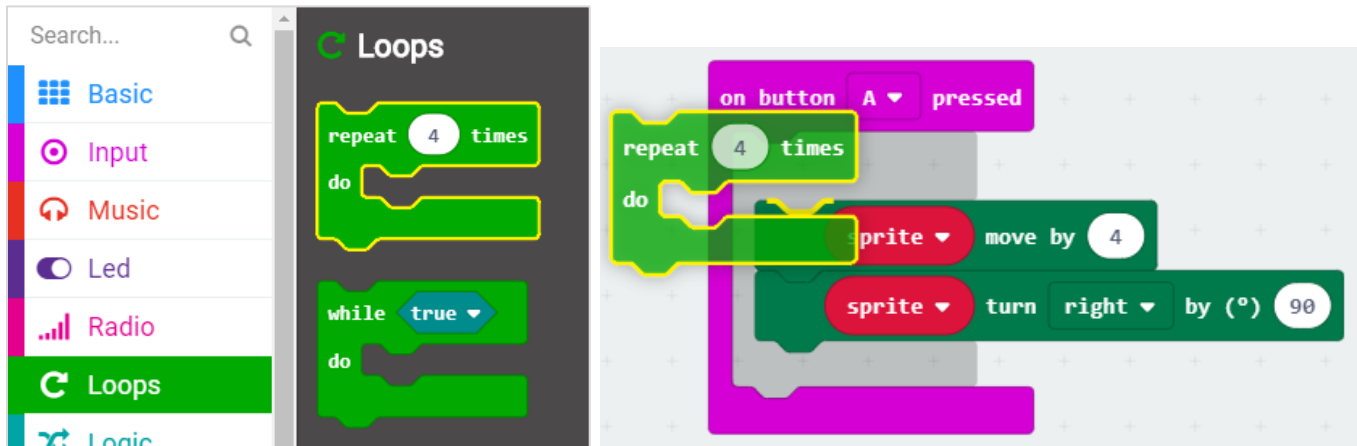
7. To make the sprite turn to walk a square, change the 'turn right by' degrees from 45 to 90. For now, it's OK to leave the sprite turning right instead of left as you did in your pseudocode.



The 'repeat' block

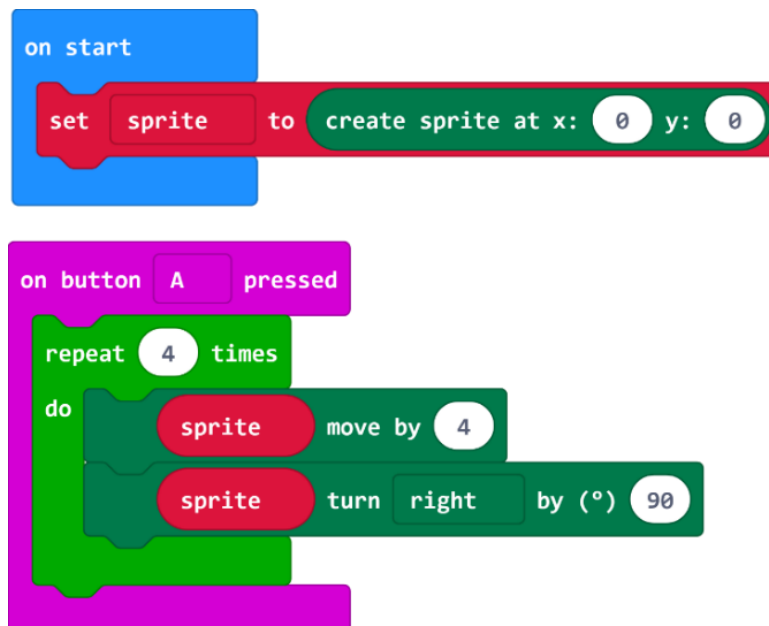
Following our pseudocode, you could add three more Move and Turn blocks to make our sprite walk a square, but there is an easier, more efficient way to code this! By using a Repeat loop.

8. Select the Loops category in the Toolbox. Drag the repeat block to the coding Workspace and place it around the sprite 'move by' and 'turn' blocks.



Notice that the 'repeat' block contains a default value of 4. This means that it will repeat whatever blocks of code it contains four times.

Your code should look like:

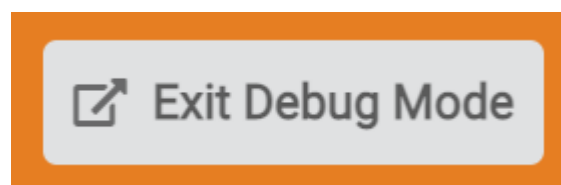
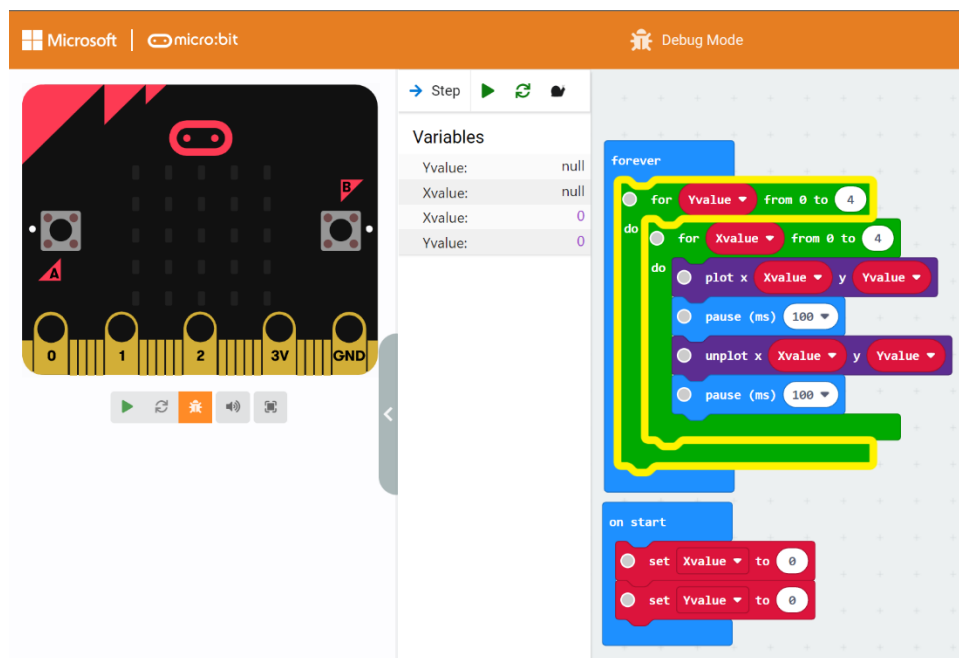
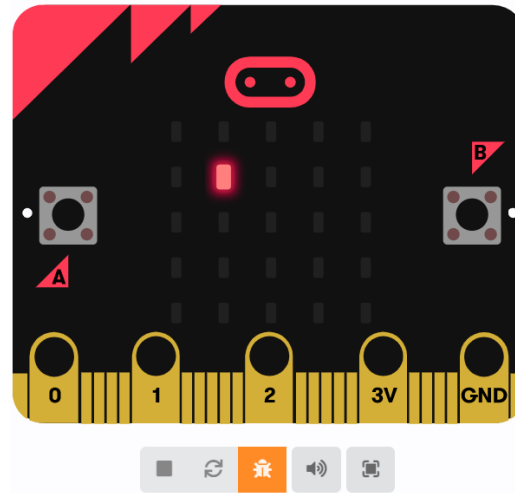


Go ahead and run the program. Make the sprite move by pressing button A in the simulator.

What happened? Did you see the sprite move? No? Why? Because it happens so quickly, you can't see the sprite appear.

Use Debug Mode

A helpful feature of Microsoft MakeCode is *Debug Mode*. Select the bug icon under the micro:bit simulator. This will halt the execution (running) of the program. Press the Step button to step through your program. Debug Mode will highlight parts of your code so you can see step by step which line of code is being processed.



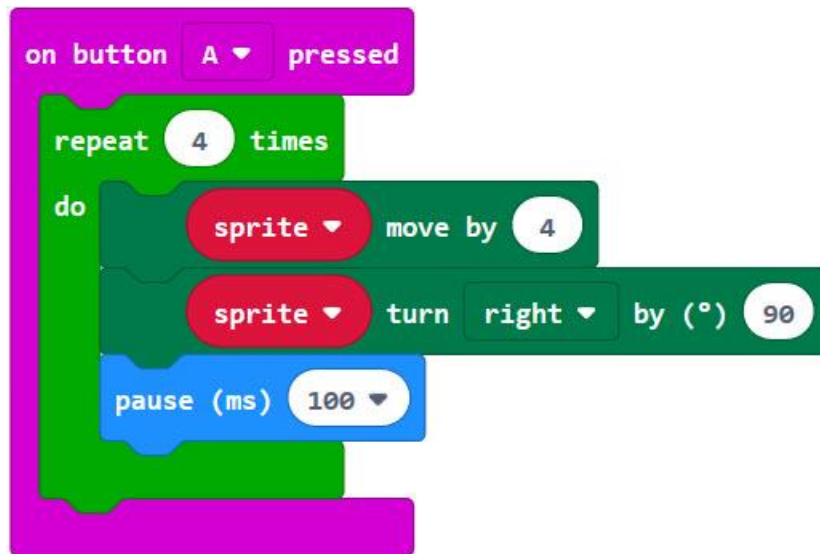
Now, run your program several more times. Do you see the different lines of your code highlighted as the program runs? Do you see the sprite move?

Add a pause

So, the code is running and the sprite is moving! Sometimes we forget just how fast computers are. So that you can see the sprite move even in "regular" mode, let's add a pause to our program right after each time the sprite moves. This will give our human eyes a chance to see it move.

Select the bug icon again to turn off Debug Mode.

9. From the Basic Toolbox category, drag a 'pause' block to the coding window and add it to our 'repeat' block right after the 'turn right by' block.



Download and run your program on the micro:bit. Now, we can see the sprite move. It still moves pretty quickly, but at least we can see it move.

Optional Mod

If there is time, experiment with changing the Pause value, the number of times to Repeat, or the number of spaces to move the Sprite to see how these changes affect your program.

Coding activity 2: Code a traveling light with 'for' loops

Now, you'll move on to code with the 'for' loop block. The 'for' block is useful when you have a variable in your loop that you want to change by a fixed amount within a specific range each time through a loop. What does this mean? Let's look at an example.

Let's make an LED light move across the entire micro:bit display from left to right, top row to bottom row.

The pseudocode

Your pseudocode for the first row might look like:

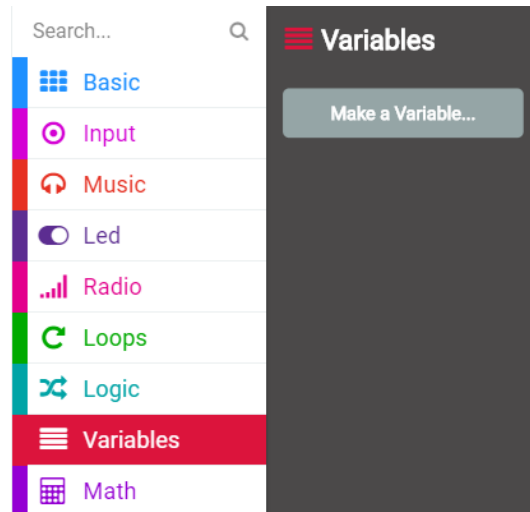
```
Turn led (x:0, y:0) on
Pause
Turn led (x:0, y:0) off
Pause
Turn led (x:1, y:0) on
Pause
Turn led (x:1, y:0) off
Pause
Turn led (x:2, y:0) on
Pause
Turn led (x:2, y:0) off
Pause
Turn led (x:3, y:0) on
Pause
Turn led (x:3, y:0) off
Pause
Turn led (x:4, y:0) on
Pause
Turn led (x:4, y:0) off
```

That's a lot of code, and most of it repeats. It's perfect for a loop!

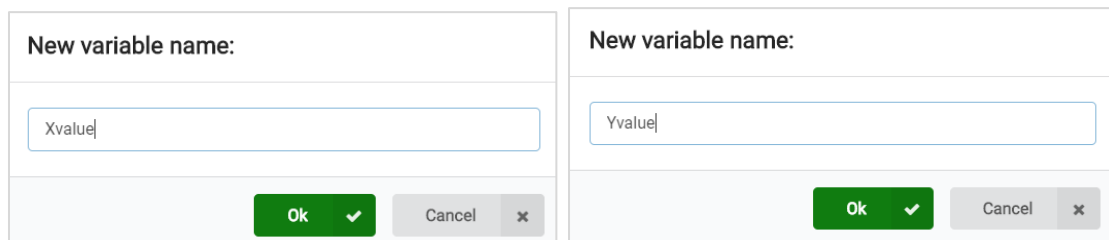
Create variables to hold the x and y position

The first thing you'll want to do is create some Variables to hold the X and Y position values.

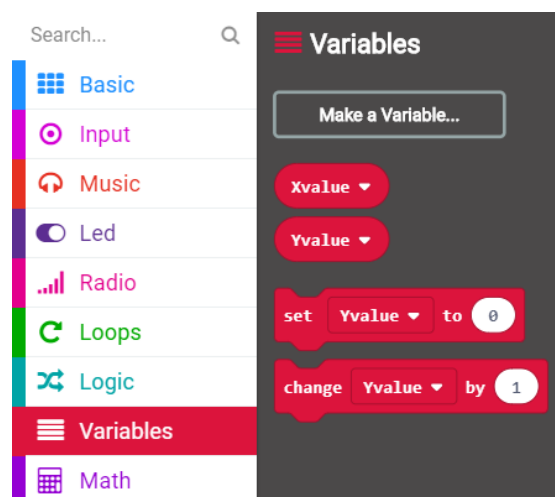
1. Create a new project and name it something like: **Traveling light**. Then, select the Variables Toolbox drawer and select the **Make a Variable** button.



2. Name the new variable something like: **Xvalue**. Then, create another variable and name it something like: **Yvalue**.

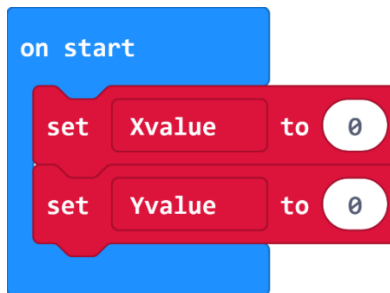


Notice that these variable blocks now appear in the Variables Toolbox drawer.



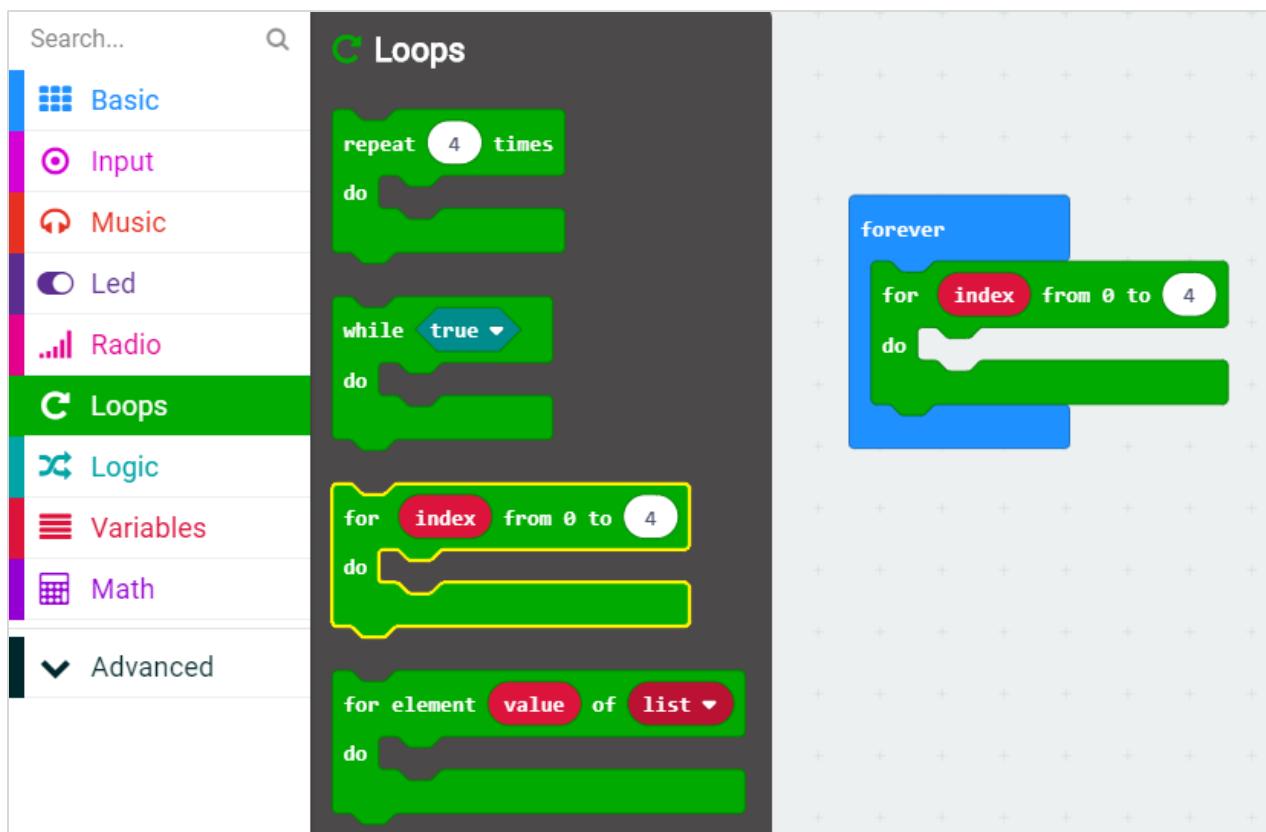
Now, you need to set the starting value for the x and y variables to be 0.

- From the Variable Toolbox drawer, drag two 'set (variable)' blocks onto the Workspace and drop them into the 'on start' block. Then, in one of the 'set (variable)' blocks, use the dropdown menu to select the 'Xvalue' variable.

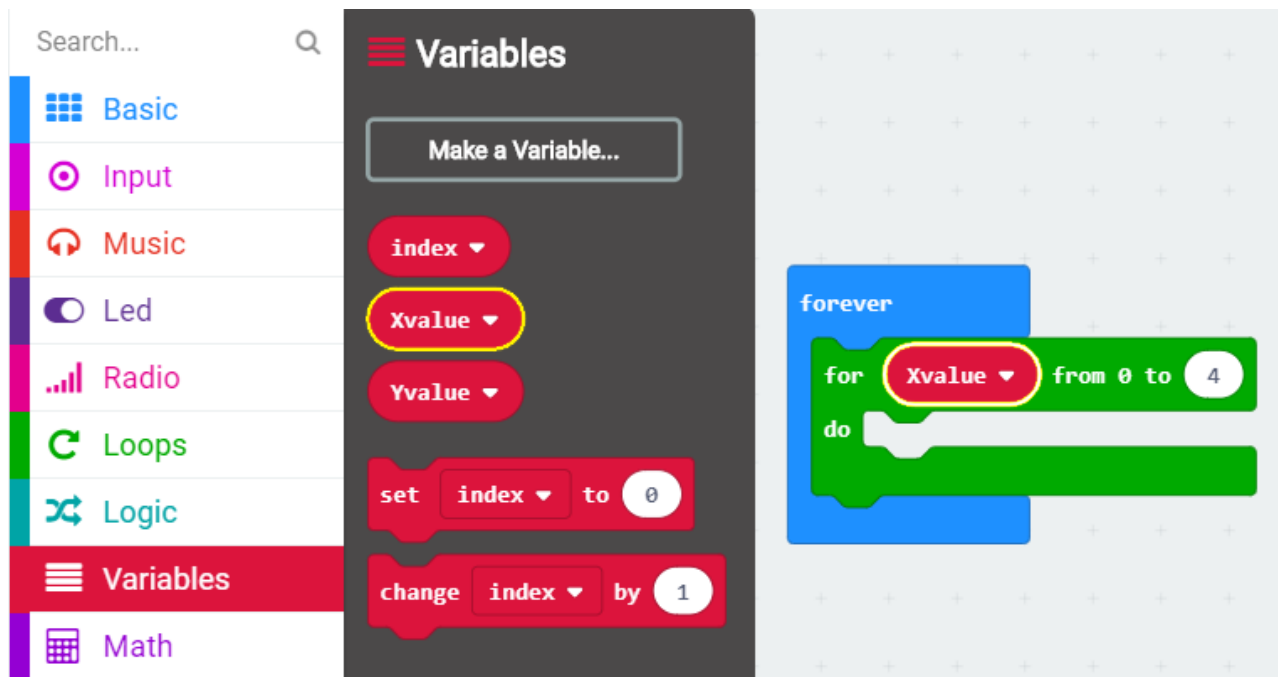


Code the loop for the x values

- From the Loops Toolbox drawer, drag a 'for' block to the coding Workspace and drop it into the 'forever' loop.

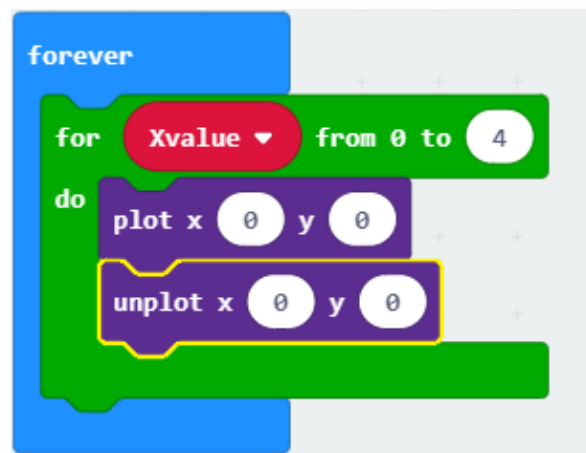


5. Instead of the default 'index' variable, you're going to use the 'Xvalue' variable that we created. From the Variables Toolbox drawer, drag the 'Xvalue' variable block out onto the Workspace and drop it into the 'for loop', replacing the 'index' block.

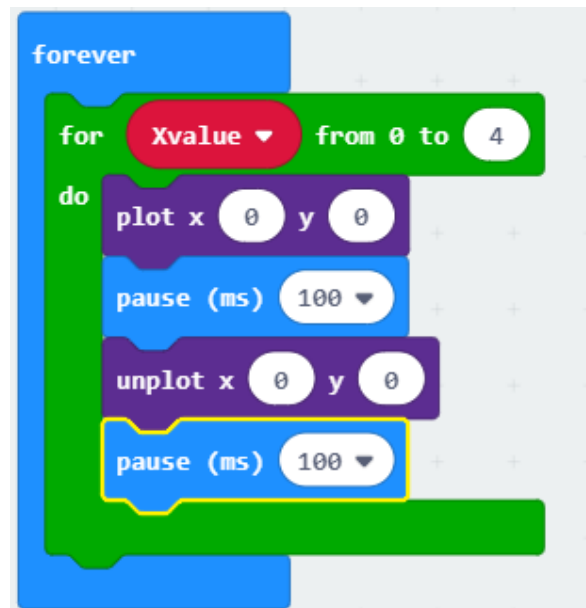


In this way, each time we iterate through the 'for' loop, our 'Xvalue' variable will increment its value—starting from 0 and going up to 4.

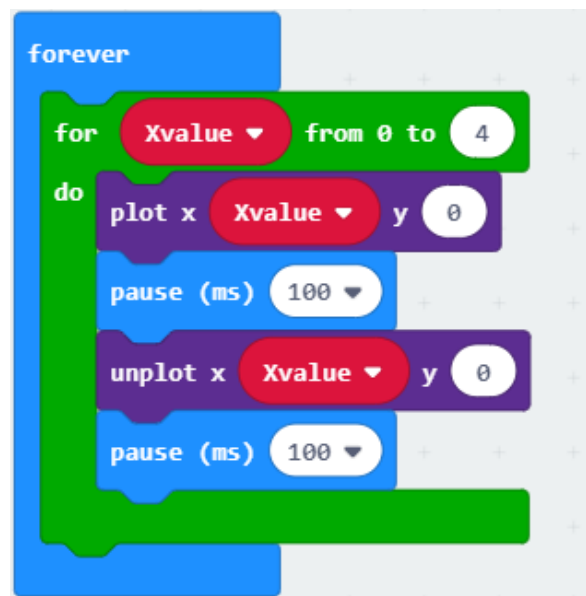
6. Plot and unplot the LED lights to turn them on and off. From the Led Toolbox drawer, drag a 'plot' block and an 'unplot' block to the coding Workspace. Drop both into the 'for' loop.



- From the Basic Toolbox drawer, drag two 'pause' blocks to the coding Workspace. Drop them in the 'for' loop—one after the 'plot' block, and one after the 'unplot' block. This will slow things down a bit so we can see the lights turning on and off.



- Change the value of the x coordinates in the 'plot' and 'unplot' blocks to the x coordinate value from the 'for' loop. From the Variables Toolbox drawer, drag two 'Xvalue' variable blocks onto the Workspace and drop one each into the x coordinate of the 'plot' block, and the x coordinate of the 'unplot' block.

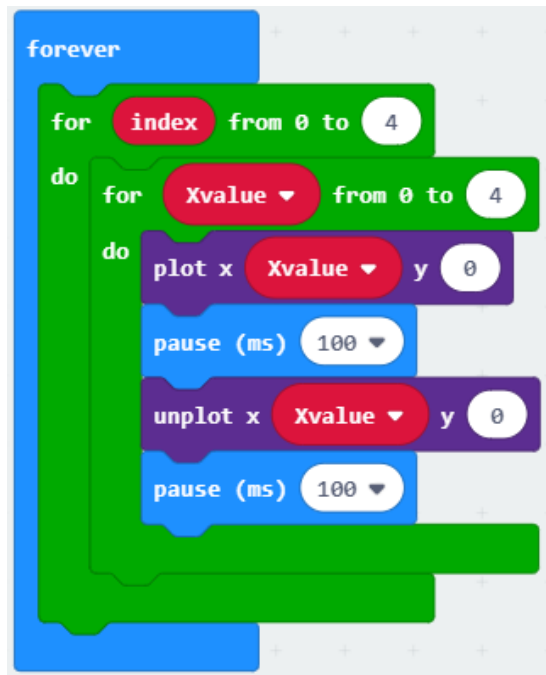


Now, you should see a light moving from left to right along the top row of the micro:bit simulator!

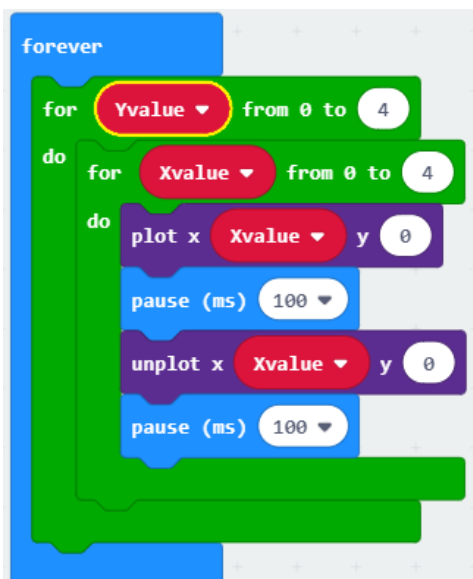
Code the loop for the y values

To make your pattern continue through all the LEDs, you can change the value of the Y coordinate as well. To do this efficiently using the fewest lines of code, you can put a loop inside a loop. Loops inside other loops are known as *nested loops*.

9. Drag out another 'for' loop block from the Loops Toolbox drawer and place it around our original 'for' block, all within the 'forever' block.



10. From the Variables Toolbox drawer, drag out the 'Yvalue' variable block and drop into the outer 'for' loop, replacing the default 'index' variable.



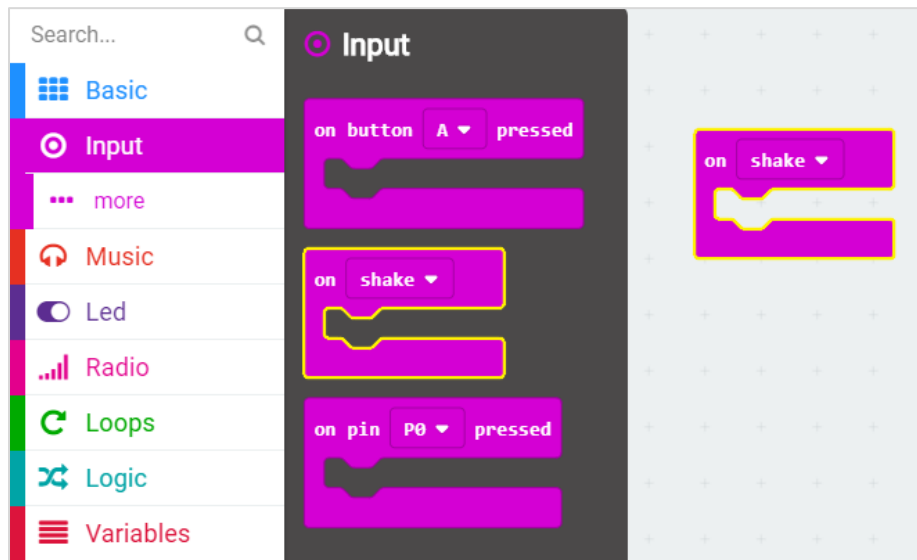
11. Change the value of the y coordinates in the 'plot' and 'unplot' blocks to this same variable (just like we did for the x coordinate earlier).

Coding activity 3: Code a micro:bit alarm with a 'while' loop

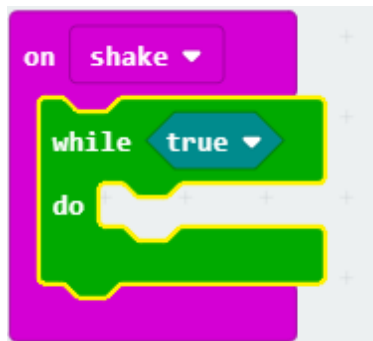
The 'while' block is useful when you want your program to loop until a certain event happens or a different condition is met. For example, maybe you want an alarm to sound if someone shakes your micro:bit. In order to turn the alarm off, you press the button A. Until you press the button, the alarm should continue to sound. You can use a 'while' block with a nested 'repeat' block.

The 'while' loop

1. Create a new project in MakeCode and name it: **Alarm Clock**. Delete the default 'on start' and 'forever' blocks from the coding Workspace. Then, from the Input Toolbox drawer, drag an 'on shake' block onto the Workspace.



2. From the Loops Toolbox drawer, drag a 'while' block to the Workspace and drop it inside the 'on shake' block.

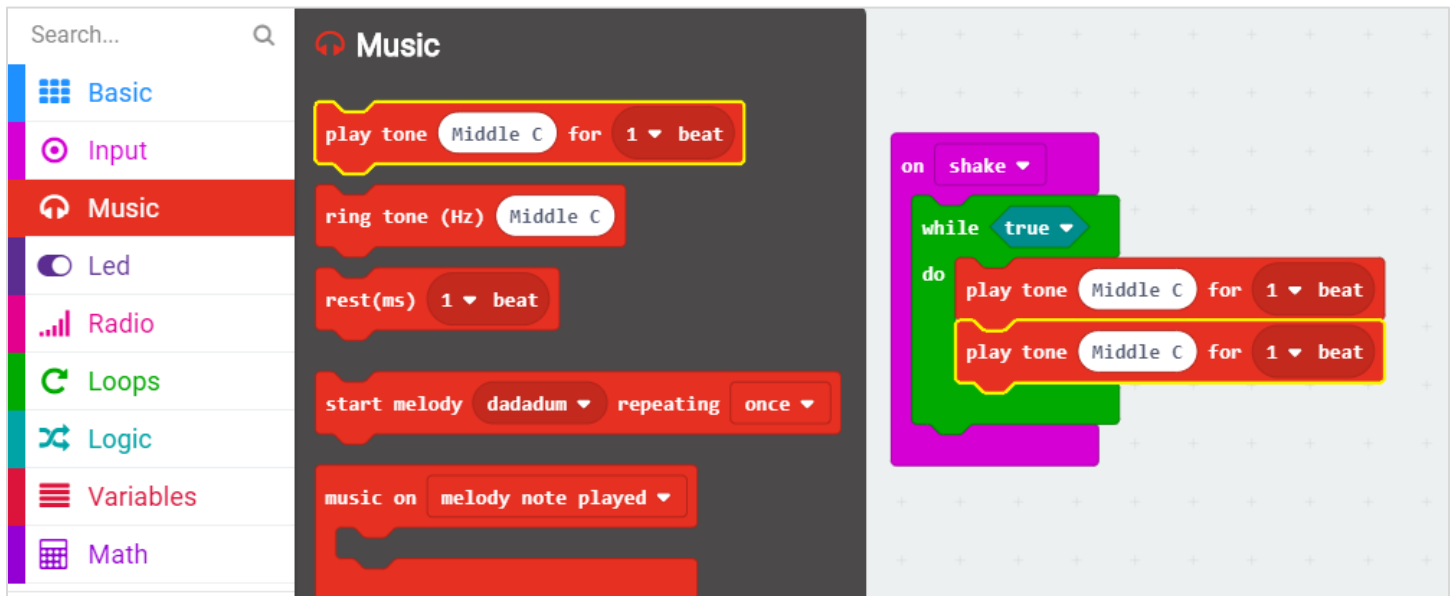


Notice that there is a <true> condition attached to the 'while' loop. Recall from the previous lesson that a conditional expression may evaluate to true or false. In this case, the default <true> block always evaluates to true, so this 'while' loop will repeat forever!

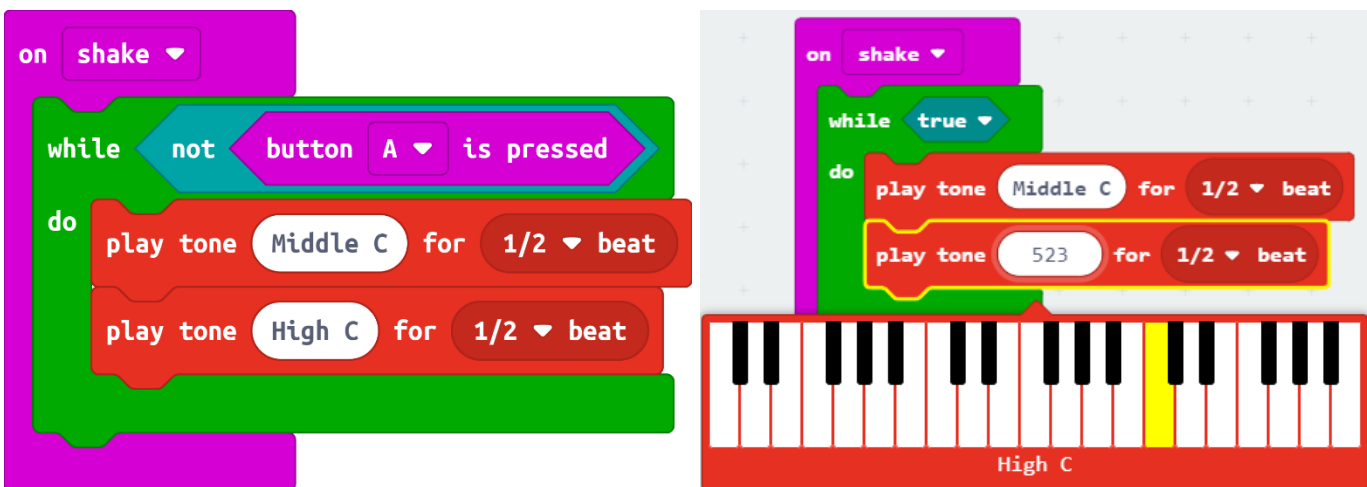
Alarm 'on shake'

You'll come back to the 'while' loop condition, but first code your alarm sound.

3. From the Music Toolbox drawer, drag two 'play tone' blocks to the coding Workspace and drop them inside the 'while' loop.



4. In the 'play tone' blocks, use the dropdown menu to change the beat value to '1/2' a beat. In the second 'play tone' block, change the tone from 'Middle C', to 'High C'.



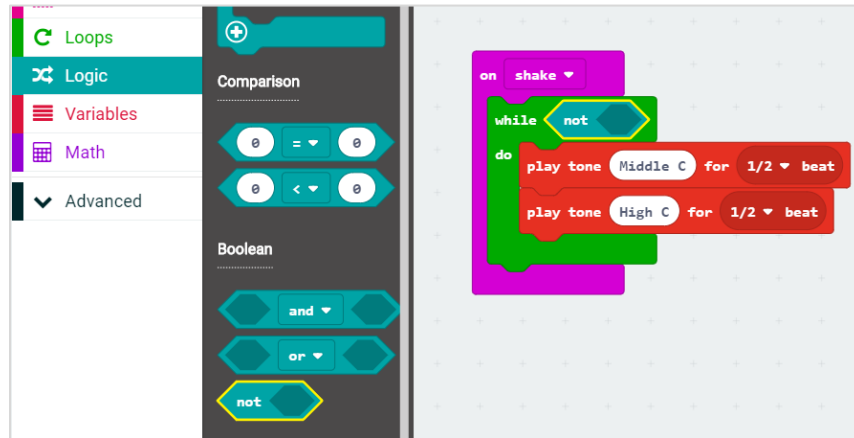
Try your code in the Simulator. What happens when you shake the micro:bit?

Your alarm goes off, and because the 'while' loop repeats continuously, there's no way to turn off your alarm!

Turn off alarm when press button

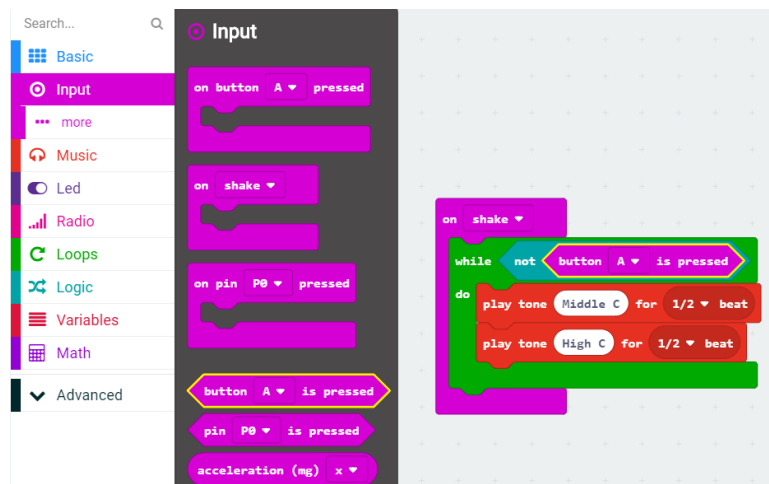
Add a condition to your 'while' loop that will turn off the alarm when the user presses a button.

- From the Logic Toolbox drawer, scroll down to the Boolean section. Drag a '**not**' hexagon block to the workspace. Drop it in the '**while**' loop, replacing <true>.



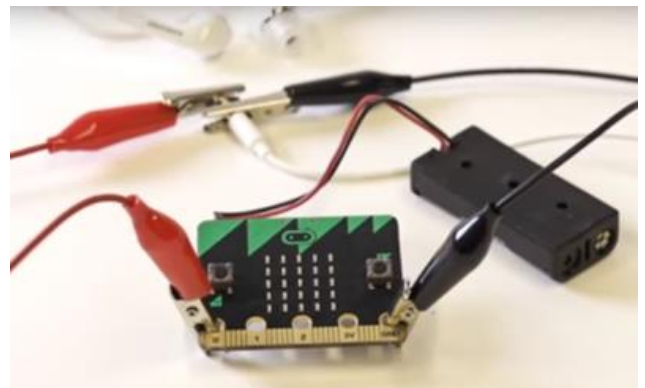
- From the Input Toolbox drawer, drag the '**button A is pressed**' hexagon-shaped block onto the Workspace and drop into the '**not**' block in the '**while**' loop.

Hint: Make sure you hover the '**button A is pressed**' hexagon over the '**not**' hexagon until only the empty hexagon shows the yellow outline so you don't replace the entire '**not**' hexagon.



Now, your 'while' loop will only repeat as long as button A has not been pressed.

Test the code in the simulator.



Lesson C: Get loopy!

Activity: Get loopy! project

In this project, you will create a program with loops, variables, and parameters, then design and build an object that uses the micro:bit program and sound, display, and motion in some way.

Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- Uses at least three different loops in a meaningful way
- Uses unique variable names that clearly describe what the variable values hold
- Uses sound, display, and motion in a way that's integral to the program
- The program compiles and runs as intended and includes meaningful comments in the code
- Provide the written Reflection Diary entry (which we'll talk about after you complete your project)

The design thinking process:

1. **Empathize** by learning more about your target audience.
2. **Define**: Understand and identify your audience's problems or needs.
3. **Ideate**: Brainstorm several possible creative solutions.
4. **Prototype**: Construct rough drafts or sketches of your ideas.
5. **Test** your prototype solutions until you come up with the final version.

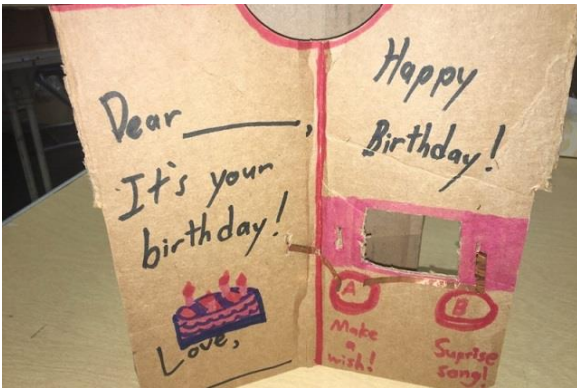
Use this space to start your design thinking process:

Project ideas

- Create an animated gif (looping image that changes) and add music that matches.
- Create an animation that repeats for one of the melodies included in MakeCode (like Happy Birthday).
- Create different animations that run when different buttons are pressed.
- Create an alarm that includes sound and images. What will set the alarm off? What will make the alarm stop sounding?
- Use servo motors to create a creature that dances and changes its expression while a song plays.

Project examples

A musical birthday card



micro:bit hat man project

This project uses the micro:bit light sensor to display a happy face when it is sunny, and a frowning face when it is dark. The micro:bit is connected to a servo mounted on the inside of the container, and the smile and frown are attached to plastic coffee stirrers with tape and hot glue. Check it out in action at youtu.be/Xvybu_T5IL8.



Inside view – Check it out it in action at youtu.be/ZfKgFQjygQQ.

Project scoring rubric

Assessment elements	1	2	3	4
No loops are implemented.	No loops are implemented.	At least one loop is implemented in a meaningful way.	At least two loops are implemented in a meaningful way.	At least three different loops are implemented in a meaningful way.
None of the variable names clearly describe what information values the variables hold.	None of the variable names clearly describe what information values the variables hold.	Few variable names are unique or clearly describe what information values the variables hold.	The majority of variable names are unique and clearly describe what information values the variables hold.	All variable names are unique and clearly describe what information values the variables hold.
None of the required elements are used.	None of the required elements are used.	Uses only one of the required elements in a way that is integral to the program.	Uses only two of the required elements in a way that is integral to the program.	Uses sound, display, and motion in a way that is integral to the program.
micro:bit program lacks three or more of the required elements.	micro:bit program lacks three or more of the required elements.	micro:bit program lacks two of the required elements.	micro:bit program lacks one of the required elements.	micro:bit program: ■ Uses loops in a way that is integral to the program ■ Compiles and runs as intended ■ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- Explain how you decided on your particular “loopy” idea. What brainstorming ideas did you come up with?
- What type of loop did you use: For, While, or Repeat?
- What was something that was surprising to you about the process of creating this program?
- Describe a difficult point in the process of designing this program and explain how you resolved it.
- What feedback did your beta testers give you? How did that help you improve your loop demo?
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements: ▪ Brainstorming ideas ▪ Construction ▪ Programming ▪ Beta testing

Glossary of key terms

Iteration: Code that tells a computer to repeat sets of instructions under different conditions.

Loops: A programming construct that allows for a block of code to be executed a specified number of times.

Infinite loops: A programming construct that allows for a block of code to be executed without ever stopping.

Nested loops: A programming construct that uses a loop within another loop.